`Terraform` challenge

Write a code using Terraform that deploys and achieves the following:-

1. Create an EC2 Instance on AWS Cloud using the below-mentioned ubuntu AMI and instance_type. Or, (You can use of your choice as well) ami = "ami-0461b1436c9b3c1a3" instance_type = "t2.micro"
2. Install a web server on the same EC2 Instance. The web server should listen on port 8080.
3. Create a Docker-compose file for the project: https://github.com/betawins/cricbuzz-docker-hello-world.git and point it to port 8000.
4. Make the web server point to the above service so that 8080 returns the response from the Docker service.
5. Configure to Save the Logs in the location /var/log/cb.log
6. Write the Log rotate configuration for the above file

Terraform Challenge — Cricbuzz Docker Hello World

Technologies: AWS EC2, Terraform, Ubuntu 24.04, Docker, Docker Compose, Nginx, Logrotate

AWS Region: `us-east-1`

Project status: Successfully deployed and verified

---

1. Project Overview

1.1 Objective

The objective of this challenge was to use Terraform to create an EC2 instance on AWS and deploy a web application using Docker Compose.

A web server had to listen on port `8080` and forward incoming requests to the Docker application running on port `8000`.

The project also required access logs to be saved in:

`/var/log/cb.log`

Finally, Logrotate had to be configured to rotate and manage those log files automatically.

1.2 Assignment Requirements

No.	Requirement	Implementation
1	Create an EC2 instance using Terraform	Ubuntu 24.04 on EC2
2	Install a web server listening on port 8080	Nginx
3	Deploy the Docker Compose application on port 8000	Docker Compose
4	Forward requests from port 8080 to the Docker application	Nginx reverse proxy
5	Save access logs in `/var/log/cb.log`	Nginx access log
6	Configure log rotation	Linux Logrotate

---

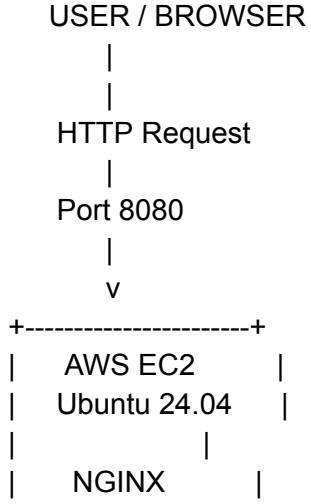
2. Architecture

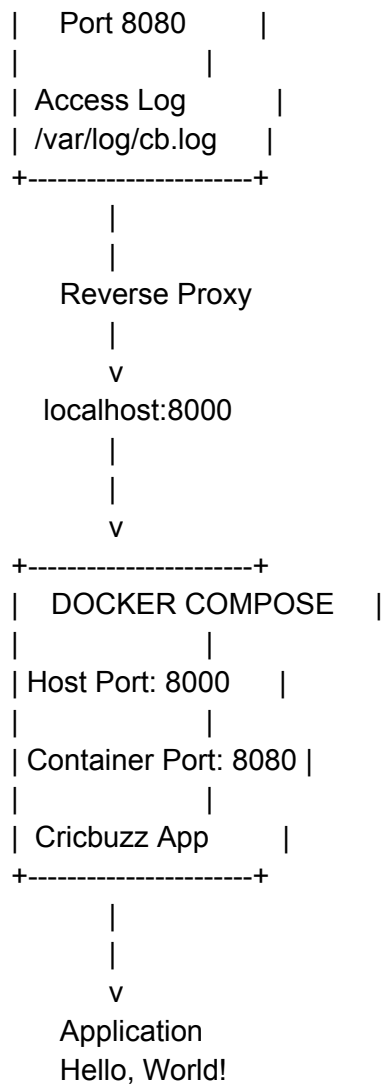
2.1 Application Architecture

The customer accesses the EC2 public IP using port `8080`.

Nginx receives the request and forwards it to the Docker application through local port `8000`. Docker forwards the request to container port `8080`.

The application's response travels back through Nginx to the customer.





2.2 Understanding the Three Ports

Three port references are important in this project.

Port	Component	Purpose
8080	Nginx	Public web access
8000	EC2 host	Local connection to Docker
8080	Docker container	Application listening port

The Docker Compose mapping is:

```
127.0.0.1:8000:8080
```

This means:

- `127.0.0.1`: Docker publishes the host port only on the EC2 loopback interface.
- First `8000`: Port on the EC2 host.
- Second `8080`: Port inside the Docker container.

Nginx listens on port `8080` and forwards requests to `http://127.0.0.1:8000`.

Important: Docker port `8000` is not exposed directly to the internet. Users access the application through Nginx on port `8080`.

---

3. AWS Environment Preparation

Before writing Terraform code, we checked the AWS account, AMI, SSH key, VPC and subnet.

3.1 Verify AWS Account

Command:

aws sts get-caller-identity

Why did we use this command?

It confirms that the AWS CLI is configured correctly and identifies which AWS account and IAM user will be used.

Result from this project:

Account: 986881030460
IAM User: DevopsAdmin

3.2 Check the Ubuntu AMI

The assignment provided this AMI:

`ami-0461b1436c9b3c1a3`

When we checked it in `us-east-1`, AWS returned:

`InvalidAMIID.NotFound`

AMI IDs are region-specific. Therefore, we retrieved a valid Ubuntu 24.04 AMI from AWS Systems Manager Parameter Store.

Command:

```
$UbuntuAmi = aws ssm get-parameter `
  --name
"/aws/service/canonical/ubuntu/server/24.04/stable/current/amd64/hvm/ebs-gp3/ami-id" `
  --region us-east-1 `
  --query "Parameter.Value" `
  --output text

Write-Host "Ubuntu AMI ID: $UbuntuAmi"
```

Verified AMI:

ami-0d6dd9e8265d48906

We verified the image:

```
aws ec2 describe-images `
  --region us-east-1 `
  --image-ids $UbuntuAmi `
  --query "Images[0].[ImageId,Name,Architecture,State]" `
  --output table
```

Result:

- Ubuntu 24.04 LTS
- x86_64 architecture
- AMI state: available

3.3 Check EC2 Key Pair

We checked the existing EC2 key pair:

```
aws ec2 describe-key-pairs `
  --region us-east-1 `
  --query "KeyPairs[].KeyName" `
  --output table
```

The available key pair was:

```
my_server
```

The private key used for SSH was:

```
C:\Users\HP\Downloads\my_server.pem
```

3.4 Check Default VPC

Command:

```
aws ec2 describe-vpcs `
  --region us-east-1 `
  --filters Name=isDefault,Values=true `
  --query "Vpcs[0].VpcId" `
  --output text
```

Result:

```
vpc-09ace864d17ee9449
```

We reused the existing default VPC rather than creating a new VPC.

3.5 Identify a Public Subnet

Command:

```
aws ec2 describe-subnets `
  --region us-east-1 `
  --filters `
    Name=vpc-id,Values=vpc-09ace864d17ee9449 `
    Name=default-for-az,Values=true `
  --query "Subnets[*].[SubnetId,AvailabilityZone,CidrBlock,MapPublicIpOnLaunch]" `
  --output table
```

Selected subnet:

Setting	Value
Subnet ID	`subnet-00444f9336cffeedf`
Availability Zone	`us-east-1a`
CIDR	`172.31.0.0/20`
Auto-assign public IP	Enabled

3.6 Identify the Public IP for SSH

Command:

```
$MyIP = (Invoke-RestMethod "https://checkip.amazonaws.com").Trim()
Write-Host "My public IP: $MyIP"
```

Result:

```
124.123.159.234
```

We allowed SSH only from:

```
124.123.159.234/32
```

This is safer than allowing SSH from every internet address.

---

4. Create the Terraform Project

4.1 Create the Project Directory

We created a separate directory so that this challenge would not mix with the previous Terraform assignment.

```
New-Item -ItemType Directory -Force -Path "C:\Users\HP\terraform-cricbuzz-challenge"
```

```
cd "C:\Users\HP\terraform-cricbuzz-challenge"
```

4.2 Project Structure

We created these files:

```
terraform-cricbuzz-challenge/
│
├── .gitignore
├── versions.tf
├── provider.tf
├── variables.tf
├── main.tf
├── outputs.tf
```

```
├── user_data.sh
└── .terraform.lock.hcl
```

4.3 Purpose of Each File

File	Purpose
`versions.tf`	Defines Terraform and provider version requirements
`provider.tf`	Configures the AWS provider
`variables.tf`	Stores reusable input values
`main.tf`	Creates EC2 and Security Group resources
`outputs.tf`	Displays instance information after deployment
`user_data.sh`	Installs and configures Docker, Nginx and Logrotate
`.gitignore`	Prevents sensitive Terraform files from being committed
`.terraform.lock.hcl`	Records the selected provider version and checksums

---

5. Complete Terraform Source Code

5.1 versions.tf

```
terraform {
  required_version = ">= 1.5.0"

  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 6.0"
    }
  }
}
```

Explanation:

- `terraform`: Defines Terraform configuration.

- `required_version`: Specifies the minimum Terraform version.
- `required_providers`: Declares the providers required by the project.
- `source`: Identifies the AWS provider from HashiCorp.
- `version`: Restricts the provider to compatible 6.x releases.

During initialization, Terraform installed AWS provider version `6.71.0`.

5.2 provider.tf

```
provider "aws" {
  region = var.aws_region
}
```

Explanation:

The AWS provider connects Terraform to AWS.

The region comes from the `aws_region` variable.

Terraform uses the AWS CLI credentials configured on the local machine.

5.3 variables.tf

```
variable "aws_region" {
  default = "us-east-1"
}

variable "ami_id" {
  default = "ami-0d6dd9e8265d48906"
}

variable "instance_type" {
  default = "t3.micro"
}

variable "key_name" {
  default = "my_server"
}

variable "vpc_id" {
  default = "vpc-09ace864d17ee9449"
}

variable "subnet_id" {
```

```
  default = "subnet-00444f9336cffeedf"
}

variable "ssh_cidr" {
  default = "124.123.159.234/32"
}
```

Explanation:

Variables keep configuration values in one place.

For example:

```
variable "instance_type" {
  default = "t3.micro"
}
```

This allows us to change the EC2 instance type without modifying the main resource definition.

Why did we use t3.micro instead of t2.micro?

Initially, Terraform attempted to create a `t2.micro` instance. AWS rejected it with:

```
InvalidParameterCombination:
The specified instance type is not eligible for Free Tier.
```

We queried the eligible instance types:

```
aws ec2 describe-instance-types `
  --region us-east-1 `
  --filters Name=free-tier-eligible,Values=true `
  --query "InstanceTypes[*].[InstanceType]" `
  --output table
```

AWS returned several eligible options, including `t3.micro`.

We changed the instance type to `t3.micro`.

We also configured Standard CPU credit mode in `main.tf` to avoid surplus CPU-credit charges associated with Unlimited mode.

Remember that AWS eligibility depends on account age, billing terms and usage. A displayed eligible instance type does not guarantee that the complete deployment will be free.

5.4 main.tf

This file creates two AWS resources:

1. Security Group
2. EC2 instance

```
# Create Security Group
resource "aws_security_group" "cricbuzz_sg" {
  name        = "cricbuzz-web-sg"
  description = "Allow SSH and application traffic"
  vpc_id      = var.vpc_id

  # SSH access from your public IP only
  ingress {
    description = "SSH"
    from_port   = 22
    to_port     = 22
    protocol    = "tcp"
    cidr_blocks = [var.ssh_cidr]
  }

  # Public access to Nginx
  ingress {
    description = "Web application"
    from_port   = 8080
    to_port     = 8080
    protocol    = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }

  # Allow outbound traffic
  egress {
    from_port   = 0
    to_port     = 0
    protocol    = "-1"
    cidr_blocks = ["0.0.0.0/0"]
  }

  tags = {
    Name = "cricbuzz-web-sg"
  }
}

# Create EC2 instance
resource "aws_instance" "cricbuzz" {
  ami           = var.ami_id
```

```
  instance_type = var.instance_type
  key_name      = var.key_name

  subnet_id                   = var.subnet_id
  associate_public_ip_address = true

  vpc_security_group_ids = [
    aws_security_group.cricbuzz_sg.id
  ]

  user_data = file("${path.module}/user_data.sh")

  user_data_replace_on_change = true

  # Require IMDSv2
  metadata_options {
    http_tokens = "required"
  }

  # Standard CPU credit mode
  credit_specification {
    cpu_credits = "standard"
  }

  # Encrypted root disk
  root_block_device {
    volume_size           = 10
    volume_type           = "gp3"
    encrypted             = true
    delete_on_termination = true
  }

  tags = {
    Name = "Terraform-Cricbuzz-Server"
  }
}
```

Understanding the Security Group

A Security Group acts as a virtual firewall for the EC2 instance.

Port	Protocol	Source	Purpose

22	TCP	`124.123.159.234/32`	SSH administration
8080	TCP	`0.0.0.0/0`	Public website
All outbound	All	`0.0.0.0/0`	Outbound connectivity

Port `8000` is not opened publicly because Docker binds it to `127.0.0.1`.

Understanding the EC2 Resource

- `ami`: Selects Ubuntu 24.04.
- `instance_type`: Creates a `t3.micro` EC2 instance.
- `key_name`: Uses the existing `my_server` key pair.
- `subnet_id`: Launches EC2 in the selected public subnet.
- `associate_public_ip_address`: Assigns a public IP.
- `vpc_security_group_ids`: Attaches the Security Group.
- `user_data`: Executes the installation script on first boot.
- `metadata_options`: Requires IMDSv2.
- `credit_specification`: Uses Standard CPU credit mode.
- `root_block_device`: Creates an encrypted 10 GB gp3 disk.

5.5 outputs.tf

```
output "instance_id" {
  value = aws_instance.cricbuzz.id
}

output "public_ip" {
  value = aws_instance.cricbuzz.public_ip
}

output "website_url" {
  value = "http://${aws_instance.cricbuzz.public_ip}:8080"
}
```

Explanation:

After Terraform creates the EC2 instance, these outputs display:

- EC2 instance ID
- Public IP address

- Website URL

This avoids having to manually search for the instance information in the AWS Console.

5.6 .gitignore

```
.terraform/
*.tfstate
*.tfstate.*
*.tfvars
*.tfplan
crash.log
```

Why is this important?

Terraform state files and variable files can contain sensitive information. The `.gitignore` file helps prevent accidental commits of these files.

The provider lock file `.terraform.lock.hcl` should normally be committed to Git.

---

6. EC2 User Data — Automated Installation

6.1 What Is User Data?

EC2 User Data allows us to execute a startup script when an instance is launched.

Instead of manually connecting to EC2 and installing every package, we provide a script through Terraform.

In this project, the User Data script performs these operations:

1. Updates Ubuntu packages.
2. Installs Nginx.
3. Installs Docker and the Docker Compose plugin.
4. Creates a Docker Compose file.
5. Starts the Docker application.
6. Configures Nginx as a reverse proxy.
7. Creates the access log.
8. Configures Logrotate.
9. Starts and verifies the services.

6.2 user_data.sh

The following is the deployment script, including the Logrotate permission correction identified during testing.

```
#!/bin/bash
set -euxo pipefail

export DEBIAN_FRONTEND=noninteractive

# 1. Update Ubuntu
apt-get update -y

# 2. Install Nginx and required packages
apt-get install -y ca-certificates curl nginx logrotate

# 3. Add Docker official repository
install -m 0755 -d /etc/apt/keyrings

curl -fsSL https://download.docker.com/linux/ubuntu/gpg \
  -o /etc/apt/keyrings/docker.asc

chmod a+r /etc/apt/keyrings/docker.asc

. /etc/os-release

echo "deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/docker.asc]
https://download.docker.com/linux/ubuntu ${UBUNTU_CODENAME:-$VERSION_CODENAME}
stable" \
  > /etc/apt/sources.list.d/docker.list

# 4. Install Docker and Docker Compose
apt-get update -y

apt-get install -y docker-ce docker-ce-cli containerd.io \
  docker-buildx-plugin docker-compose-plugin

systemctl enable --now docker

# 5. Create application directory
mkdir -p /opt/cricbuzz

# 6. Create Docker Compose file
cat > /opt/cricbuzz/docker-compose.yml <<'COMPOSE'
```

```
services:
  cricbuzz:
    image: ghcr.io/infrastructure-as-code/hello-world:latest
    container_name: cricbuzz-hello-world
    restart: unless-stopped
    ports:
      - "127.0.0.1:8000:8080"
COMPOSE

# 7. Start the Docker application
cd /opt/cricbuzz

docker compose pull
docker compose up -d

# 8. Configure Nginx reverse proxy
cat > /etc/nginx/sites-available/cricbuzz <<'NGINX'
server {
    listen 8080;
    server_name _;

    access_log /var/log/cb.log;
    error_log /var/log/cb-error.log;

    location / {
        proxy_pass http://127.0.0.1:8000;

        proxy_http_version 1.1;

        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }
}
NGINX

# 9. Enable Nginx site
rm -f /etc/nginx/sites-enabled/default

ln -sfn /etc/nginx/sites-available/cricbuzz \
  /etc/nginx/sites-enabled/cricbuzz

# 10. Create access log
```

```
touch /var/log/cb.log
chown root:adm /var/log/cb.log
chmod 0640 /var/log/cb.log

# 11. Configure Logrotate
cat > /etc/logrotate.d/cb <<'LOGROTATE'
su root adm
/var/log/cb.log {
    daily
    rotate 7
    compress
    delaycompress
    missingok
    notifempty
    create 0640 root adm
    sharedscripts

    postrotate
        /usr/sbin/nginx -s reopen
    endscript
}
LOGROTATE

# 12. Validate Nginx
nginx -t

# 13. Start Nginx
systemctl enable --now nginx
systemctl reload nginx

# 14. Local application checks
curl -fsS http://127.0.0.1:8000/
curl -fsS http://127.0.0.1:8080/ || true
```

Important implementation note: The initial User Data script did not include `su root adm`. We added that setting manually to `/etc/logrotate.d/cb` after the first Logrotate debug test reported an insecure parent-directory permission error. The corrected script above includes the fix so that a future replacement instance can reproduce the working configuration.

Changing `user_data.sh` while `user_data_replace_on_change = true` is set can cause Terraform to replace the EC2 instance. Review the plan carefully before applying such a change.

---

7. Understanding Docker Compose

7.1 Why Did We Use Docker Compose?

Docker Compose lets us define and run an application container using a YAML configuration file.

Instead of entering a long `docker run` command, we describe the container configuration in `docker-compose.yml`.

7.2 Docker Compose File

The deployed file was:

```
services:
  cricbuzz:
    image: ghcr.io/infrastructure-as-code/hello-world:latest
    container_name: cricbuzz-hello-world
    restart: unless-stopped
    ports:
      - "127.0.0.1:8000:8080"
```

It was saved at:

`/opt/cricbuzz/docker-compose.yml`

Explanation

Field	Meaning
`services`	Defines application containers
`cricbuzz`	Service name
`image`	Container image to run
`container_name`	Explicit container name
`restart`	Restarts the container unless manually stopped
`ports`	Maps the host port to the container port

How the port mapping works

EC2 host: 127.0.0.1:8000
        |
        v
Docker container: 8080

Only local EC2 processes can reach the published host port.

Important note about the supplied GitHub URL

The assignment mentioned:

`https://github.com/betawins/cricbuzz-docker-hello-world.git`

In the implementation completed for this challenge, the Docker Compose file used the prebuilt GHCR image:

`ghcr.io/infrastructure-as-code/hello-world:latest`

We did not execute `git clone` of the specified `betawins` repository or build an image from that checkout. The deployed image returned the expected `Hello, World!` response and passed the health check. If the reviewer specifically requires the exact GitHub repository to be cloned and built, that is an additional requirement to address before submission.

---

8. Understanding Nginx Reverse Proxy

8.1 What Is Nginx?

Nginx is a web server that can also act as a reverse proxy.

In this project, it listens on port `8080` and forwards user requests to the Docker application on port `8000`.

8.2 Why Did We Use a Reverse Proxy?

The Docker application is bound only to localhost, not directly exposed to the internet.

Nginx provides the public entry point and passes requests to the internal Docker service.

8.3 Nginx Configuration

Location:

`/etc/nginx/sites-available/cricbuzz`

```
server {
    listen 8080;
    server_name _;

    access_log /var/log/cb.log;
    error_log /var/log/cb-error.log;

    location / {
        proxy_pass http://127.0.0.1:8000;

        proxy_http_version 1.1;

        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }
}
```

Explanation

- `listen 8080`: Nginx accepts HTTP requests on port 8080.
- `server_name _`: Acts as a default server for unmatched hostnames.
- `access_log`: Stores incoming request details in `/var/log/cb.log`.
- `error_log`: Stores Nginx errors in `/var/log/cb-error.log`.
- `location /`: Handles requests to the root and other paths.
- `proxy_pass`: Forwards requests to the Docker service.
- `proxy_set_header`: Passes information about the original request to the backend.

Request flow

```
User
 |
 | http://PUBLIC-IP:8080
 |
 v
Nginx
 |
```

| proxy_pass
|
v
127.0.0.1:8000
|
v
Docker application
|
v
Hello, World!

---

9. Understanding Logs

9.1 What Is an Access Log?

An access log records requests received by the web server.

It helps us identify:

- Who made the request.
- Which URL was requested.
- When the request happened.
- What HTTP status code was returned.
- How many bytes were sent.
- Which client or user agent made the request.

9.2 Required Log Location

The assignment specified:

`/var/log/cb.log`

We configured Nginx as follows:

access_log /var/log/cb.log;

We also created the file during EC2 setup:

touch /var/log/cb.log

chown root:adm /var/log/cb.log

chmod 0640 /var/log/cb.log

9.3 How to Read the Logs

sudo tail -n 20 /var/log/cb.log

To follow new requests continuously:

sudo tail -f /var/log/cb.log

Actual verified log entries

127.0.0.1 - - [02/Nov/2026:07:05:07 +0000] "GET / HTTP/1.1" 200 13 "-" "curl/8.5.0"

124.123.159.234 - - [02/Nov/2026:07:06:11 +0000] "GET / HTTP/1.1" 200 13 "-" "Mozilla/5.0"

124.123.159.234 - - [02/Nov/2026:07:06:33 +0000] "GET /health HTTP/1.1" 200 0 "-" "Mozilla/5.0"

What does HTTP 200 mean?

HTTP status code `200` means the request succeeded.

The `/health` endpoint returned HTTP 200 with an empty response body. This was an expected successful health response in our test.

---

10. Understanding Logrotate

10.1 What Is Logrotate?

Logrotate is a Linux utility used to manage log files automatically.

As applications run, log files continue to grow. If we keep every log in one file forever, it can consume disk space.

Logrotate solves this by:

1. Rotating old log files.
2. Creating a new log file.
3. Compressing older logs.
4. Retaining only the configured number of backups.

5. Removing older backups when they exceed the retention limit.

10.2 Why Did We Configure It?

The assignment specifically required log rotation for:

`/var/log/cb.log`

We configured daily rotation and seven retained rotations.

10.3 Final Logrotate Configuration

Location:

`/etc/logrotate.d/cb`

```
su root adm
/var/log/cb.log {
    daily
    rotate 7
    compress
    delaycompress
    missingok
    notifempty
    create 0640 root adm
    sharedscripts

    postrotate
        /usr/sbin/nginx -s reopen
    endscript
}
```

Explanation of Each Setting

Setting	Meaning
`su root adm`	Runs rotation using the specified user and group
`daily`	Rotates the log daily when due
`rotate 7`	Retains seven rotated log files
`compress`	Compresses older rotated logs

`delaycompress`	Delays compression until the next rotation
`missingok`	Does not report an error if the log is missing
`notifempty`	Does not rotate an empty log
`create 0640 root adm`	Creates a new log with the specified mode and ownership
`sharedscripts`	Runs the post-rotation script once for the log set
`postrotate`	Runs a command after rotating
`nginx -s reopen`	Tells Nginx to reopen its log files

10.4 Logrotate Troubleshooting

During the first debug test, we encountered:

error: skipping "/var/log/cb.log" because parent directory has insecure permissions
Set "su" directive in config file

This happened because Logrotate would not rotate the file using its default permissions handling.

We edited the configuration:

sudo nano /etc/logrotate.d/cb

We added:

su root adm

Then we tested the configuration again:

sudo logrotate -d /etc/logrotate.d/cb

The insecure-permissions error disappeared.

10.5 Test Logrotate

Debug test:

sudo logrotate -d /etc/logrotate.d/cb

This checks the configuration without actually rotating the file.

Force a rotation for testing:

sudo logrotate -vf /etc/logrotate.d/cb

This performs a real rotation immediately, regardless of the daily schedule.

Check the files:

sudo ls -lh /var/log/cb.log*

Actual result after the successful rotation:

-rw-r----- 1 www-data adm   0 Nov 2 07:17 /var/log/cb.log
-rw-r----- 1 root   adm 411 Nov 2 07:06 /var/log/cb.log.1

A new request was then generated:

curl http://127.0.0.1:8080/

It returned:

Hello, World!

We confirmed that Nginx wrote the new request into the fresh log file.

Important: The forced rotation was only a test. Normal Logrotate execution follows its daily schedule. Do not force rotation repeatedly during normal operation.

---

11. Terraform Deployment Process

After creating all Terraform files, we ran the Terraform commands in order.

Step 1: Initialise Terraform

terraform init

Why?

- Initialises the Terraform working directory.
- Downloads the required AWS provider.
- Prepares the backend.

- Creates `.terraform.lock.hcl`.

Result:

Terraform has been successfully initialized!

AWS provider installed:

`hashicorp/aws v6.71.0`

Step 2: Format the Code

terraform fmt -recursive

Why?

Formats Terraform files using standard indentation and layout.

Step 3: Validate the Configuration

terraform validate

Result:

Success! The configuration is valid.

Why?

Checks Terraform syntax and configuration consistency before deployment.

Step 4: Review the Plan

terraform plan

Initially, Terraform planned to create:

- One EC2 instance.
- One Security Group.

The plan showed:

Plan: 2 to add, 0 to change, 0 to destroy.

This means Terraform intended to create two resources and make no changes to existing resources.

Step 5: Apply the Configuration

Initially, AWS rejected `t2.micro` because it was not eligible for the Free Tier rules applied to the account.

We changed the instance type to `t3.micro` and configured Standard CPU credit mode.

After that, we ran:

terraform apply

Terraform displayed the execution plan and asked for confirmation.

We entered:

yes

Terraform created the instance successfully.

Final result:

Apply complete! Resources: 1 added, 0 changed, 0 destroyed.

---

12. SSH Connection and Troubleshooting

12.1 Connect to EC2

From Windows PowerShell:

ssh -i "C:\Users\HP\Downloads\my_server.pem" ubuntu@100.54.210.134

12.2 SSH Private Key Permission Issue

Initially, SSH reported:

WARNING: UNPROTECTED PRIVATE KEY FILE!
Permissions are too open.

Windows permissions on the `.pem` file were too broad.

We corrected them using:

$key = "C:\Users\HP\Downloads\my_server.pem"

icacls $key /inheritance:r

icacls $key/grant:r"($env:USERDOMAIN)\$($env:USERNAME):(R)"

After correcting the unwanted inherited permissions, SSH connected successfully.

12.3 Verify the Login

Inside EC2:

whoami

Result:

ubuntu

12.4 Docker Permission Issue

When we initially ran:

docker ps

Linux returned:

permission denied while trying to connect to the Docker API

The `ubuntu` user did not have permission to access the Docker socket directly.

We used `sudo`:

sudo docker ps

This successfully displayed the running container.

Why use sudo?

The Docker daemon socket is privileged. Using `sudo` allows the administrator-authorised Docker command to run without granting the user broad Docker group access.

---

13. Final Verification

The project was tested both from Windows PowerShell and directly inside the EC2 instance.

13.1 Test Website from Windows

Command:

```
Invoke-WebRequest `
  -Uri "http://100.54.210.134:8080/" `
  -UseBasicParsing
```

Result:

```
StatusCode        : 200
StatusDescription : OK
Content           : Hello, World!
```

This confirms that the website is accessible through public port `8080`.

13.2 Test Health Endpoint

```
Invoke-WebRequest `
  -Uri "http://100.54.210.134:8080/health" `
  -UseBasicParsing
```

Result:

```
StatusCode        : 200
StatusDescription : OK
```

13.3 Check Docker Service

Inside EC2:

```
sudo systemctl status docker --no-pager
```

Docker status:

```
Active: active (running)
```

13.4 Check the Container

sudo docker ps

Actual container:

IMAGE:
ghcr.io/infrastructure-as-code/hello-world:latest

NAME:
cricbuzz-hello-world

STATUS:
Up

PORTS:
127.0.0.1:8000->8080/tcp

The container was running successfully.

13.5 Check Docker Compose

sudo docker compose -f /opt/cricbuzz/docker-compose.yml ps

The output confirmed:

cricbuzz-hello-world
STATUS: Up

13.6 Check Nginx

sudo systemctl status nginx --no-pager

Result:

Active: active (running)

13.7 Validate Nginx Configuration

sudo nginx -t

Result:

nginx: the configuration file /etc/nginx/nginx.conf syntax is ok
nginx: configuration file /etc/nginx/nginx.conf test is successful

13.8 Check Application Logs

sudo tail -n 20 /var/log/cb.log

The log file contained successful `GET /` and `GET /health` requests.

13.9 Check Logrotate

sudo logrotate -d /etc/logrotate.d/cb

After adding the `su root adm` directive, the debug check completed without the earlier insecure-permissions error.

We also successfully executed a forced rotation and verified the rotated file.

---

14. Final AWS Resource Information

Resource	Value
AWS Account	`986881030460`
AWS Region	`us-east-1`
VPC	`vpc-09ace864d17ee9449`
Subnet	`subnet-00444f9336cffeedf`
EC2 Instance ID	`i-05c0ef38b31c5928e`
Instance Type	`t3.micro`
AMI	`ami-0d6dd9e8265d48906`
Security Group	`sg-0020ce2246eff5678`
Key Pair	`my_server`
Public IP	`100.54.210.134`
Private IP	`172.31.10.100`
Root Disk	10 GB gp3, encrypted

Website	`http://100.54.210.134:8080`

The public IP is assigned dynamically. It can change if the instance is stopped and started because this project did not allocate an Elastic IP.

---

15. Troubleshooting Summary

Problem	Reason	Solution
AMI not found	Supplied AMI was not available in the selected region	Retrieved an Ubuntu 24.04 AMI from SSM
`t2.micro` not eligible	AWS rejected the requested instance type	Changed to `t3.micro`
SSH private key rejected	Windows key-file permissions were too broad	Restricted `.pem` permissions using `icacls`
Docker permission denied	Ubuntu user could not access Docker socket directly	Used `sudo docker`
Logrotate insecure permissions	Logrotate required an explicit user/group for rotation	Added `su root adm`
Windows `sudo` command failed	Linux commands were run in PowerShell	Connected to EC2 using SSH and ran Linux commands there

---

16. Important Concepts Learned

Terraform

- Provider configuration
- Input variables
- EC2 resource
- Security Group resource
- User Data

- Terraform init
- Terraform fmt
- Terraform validate
- Terraform plan
- Terraform apply
- Terraform outputs

AWS

- AMI
- EC2
- VPC
- Subnet
- Security Group
- SSH key pair
- Public and private IP addresses
- EBS volume
- Instance types and CPU credits

Docker

- Docker image
- Docker container
- Docker daemon
- Docker Compose
- Port mapping
- Container restart policy
- Container verification

Linux and Nginx

- Ubuntu package installation
- systemd services
- Nginx installation
- Reverse proxy
- Access logs
- File permissions
- SSH troubleshooting

Log Management

- Access logs

- Logrotate
- Daily rotation
- Retention
- Compression
- Log permissions
- Debugging Logrotate

---

17. Cleanup — Destroy AWS Resources

After capturing all screenshots and assignment evidence, destroy the resources to avoid unnecessary AWS charges.

First, exit the EC2 SSH session:

exit

Return to the Terraform project folder in Windows PowerShell:

cd "C:\Users\HP\terraform-cricbuzz-challenge"

Review the resources that Terraform plans to destroy:

terraform plan -destroy

After confirming that the plan contains only this project's resources, run:

terraform destroy

Terraform will request confirmation.

Enter:

yes

Expected result:

Destroy complete!

Verify that the state is empty:

terraform state list

If no resources are listed, the Terraform-managed resources have been destroyed.

Keep the `.tf` files, `user_data.sh`, screenshots and documentation for future reference. Do not delete any separate Terraform project or remote backend resources.

Question: Explain your Terraform Docker deployment project.

Sample answer:

"I created an AWS EC2 instance using Terraform and deployed a Docker-based Hello World application.

First, I configured the AWS provider, variables, Security Group and EC2 resource in Terraform. I used Ubuntu 24.04 and installed Docker, Docker Compose and Nginx automatically through the EC2 User Data script.

I created a Docker Compose file to run the application on container port 8080 and mapped it to localhost port 8000 on the EC2 host.

Then I configured Nginx to listen on port 8080 and forward incoming requests to the Docker application on port 8000. This allowed users to access the application using the EC2 public IP and port 8080.

I configured Nginx access logs at `/var/log/cb.log` and created a Logrotate configuration to rotate logs daily, retain seven rotations and compress older files.

Finally, I tested the website and health endpoint, verified that Docker and Nginx were running, checked the access logs and tested Logrotate.

The website returned HTTP 200 with the response `Hello, World!`, so the deployment was successfully verified."

---

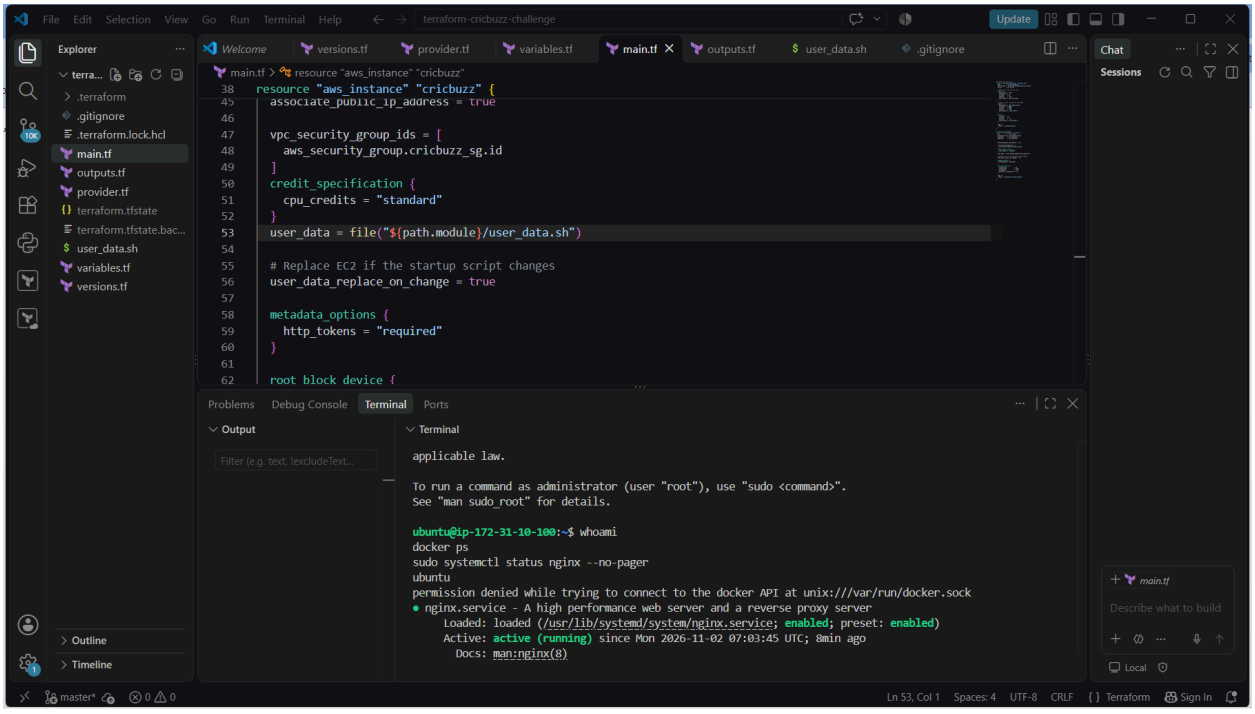
19. Final Assignment Checklist

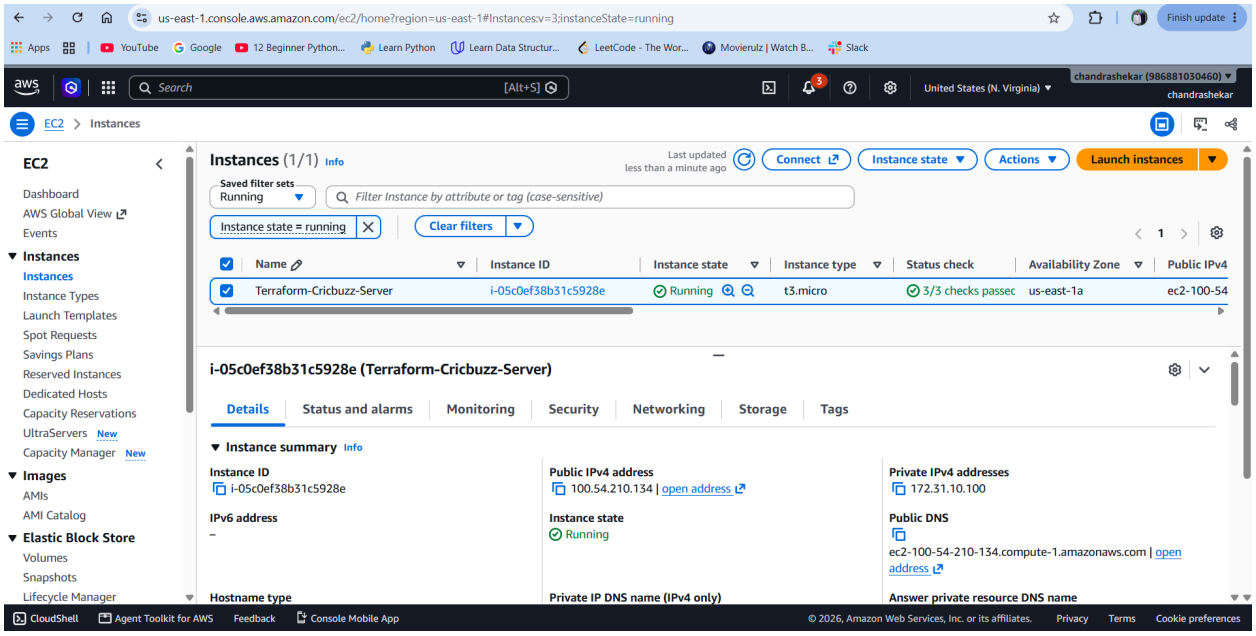
Requirement	Final result
Terraform project created	Completed
EC2 provisioned	Completed
Nginx installed	Completed
Nginx listening on port 8080	Completed

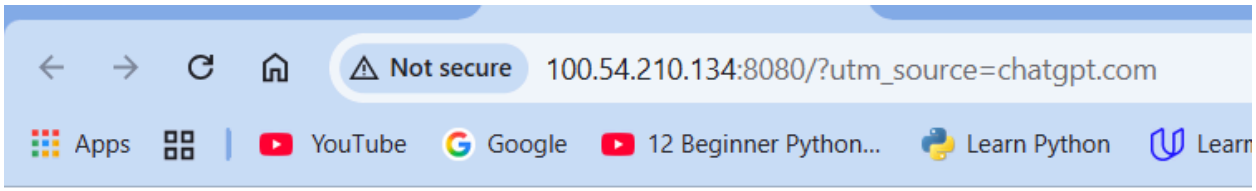
Docker installed	Completed
Docker Compose file created	Completed
Application container running	Completed
Reverse proxy configured	Completed
Access log configured	Completed
Logrotate configured	Completed
Website tested	HTTP 200
Health endpoint tested	HTTP 200
Log rotation tested	Successful

Final result: The EC2 application deployment, reverse proxy, access logging and Logrotate validation all worked successfully.

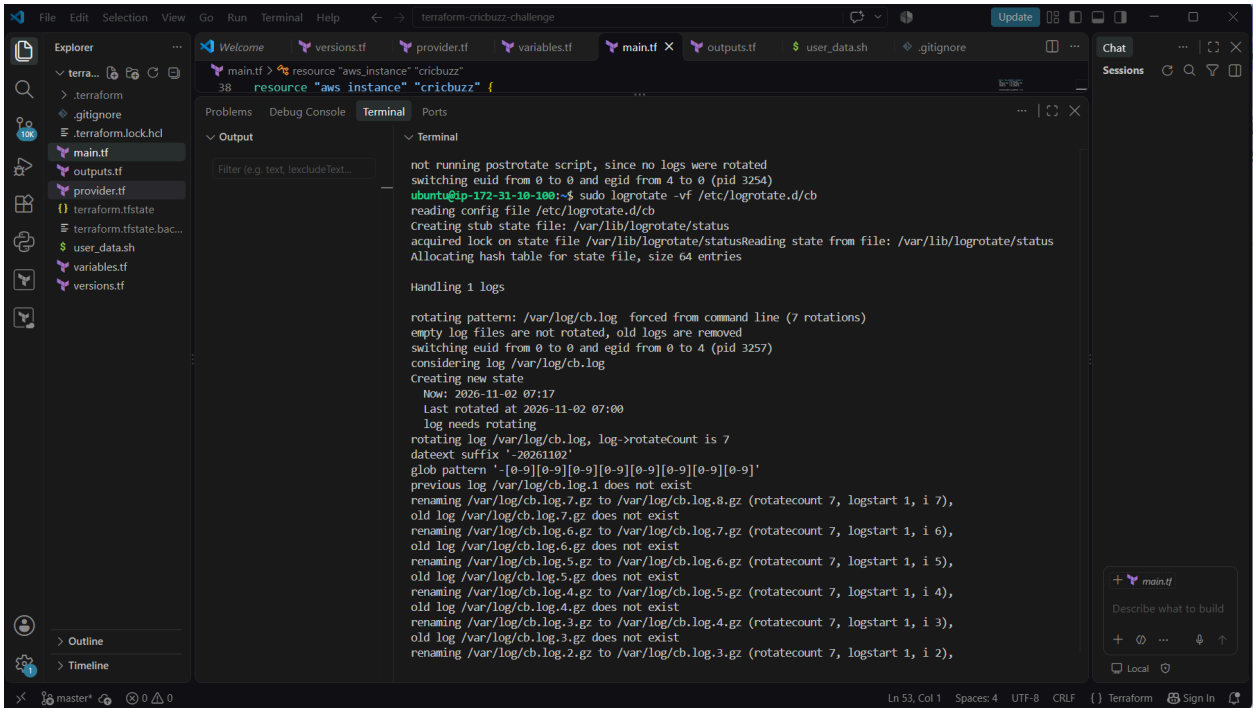
The only implementation clarification is that the deployed Compose service used the prebuilt GHCR image rather than cloning and building the exact `betawins` GitHub repository. This should be checked against the reviewer's interpretation of that requirement before submission.

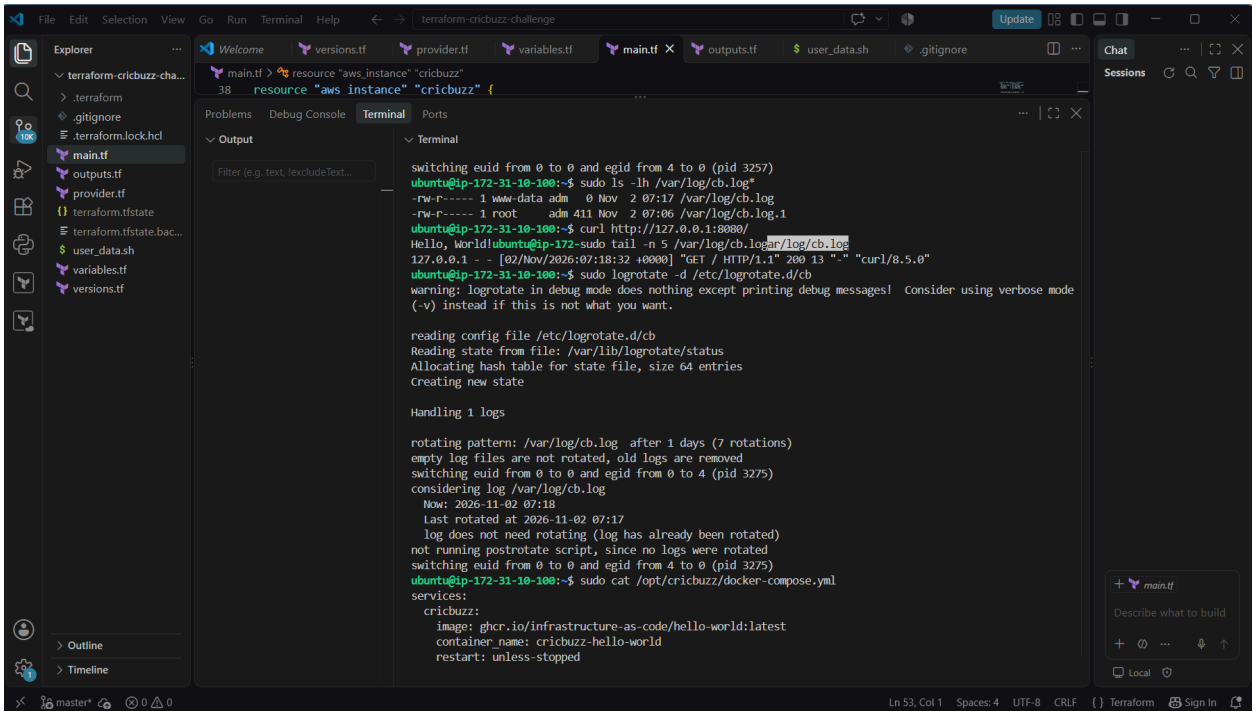


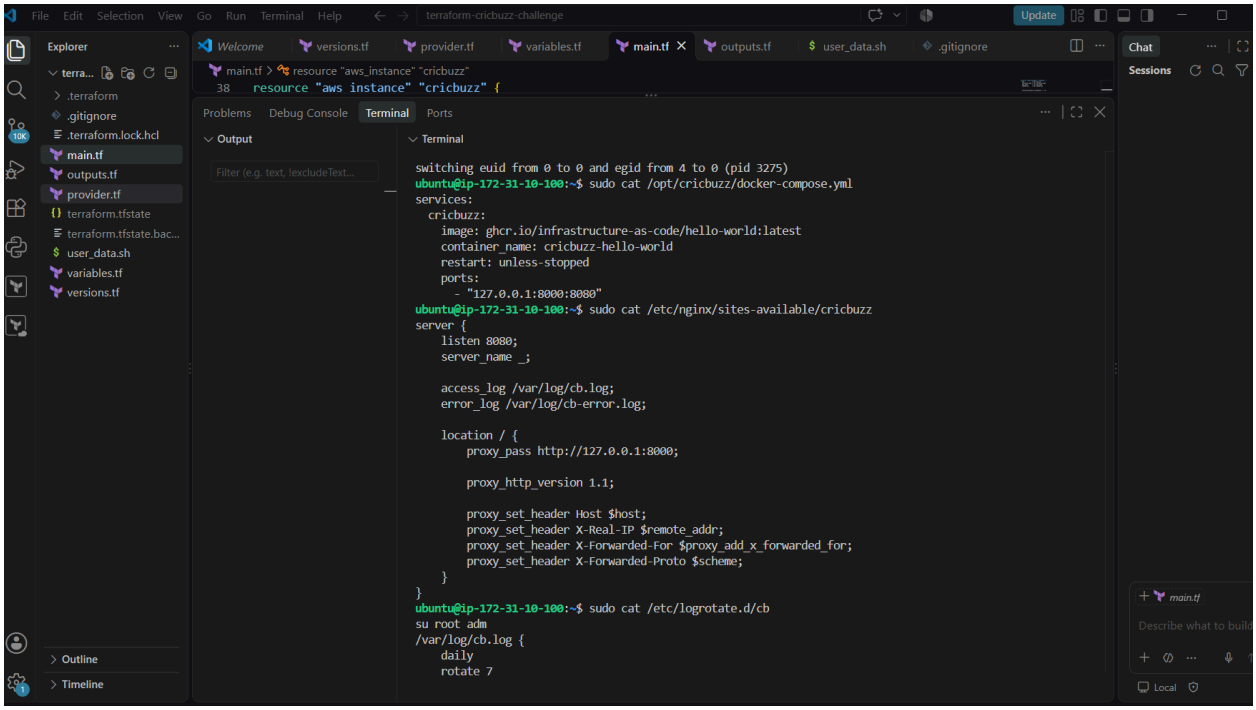

us-east-1.console.aws.amazon.com/ec2/home?region=us-east-1#Instances:v=3;instanceState=running
Finish update
Apps
YouTube
Google
12 Beginner Python...
Learn Python
Learn Data Structur...
LeetCode - The Wor...
Movierulz | Watch B...
Slack
Search
[Alt+S]
United States (N. Virginia)
chandrashekar (986881030460)
chandrashekar
EC2 > Instances
EC2
Dashboard
AWS Global View
Events
Instances
Instances
Instance Types
Launch Templates
Spot Requests
Savings Plans
Reserved Instances
Dedicated Hosts
Capacity Reservations
UltraServers New
Capacity Manager New
Images
AMIs
AMI Catalog
Elastic Block Store
Volumes
Snapshots
Lifecycle Manager
Instances (1/1) Info
Last updated less than a minute ago
Connect
Instance state
Actions
Launch instances
Saved filter sets
Running
Filter Instance by attribute or tag (case-sensitive)
Instance state = running
Clear filters
1
Name
Instance ID
Instance state
Instance type
Status check
Availability Zone
Public IPv4
Terraform-Cricbuzz-Server
i-05c0ef38b31c5928e
Running
t3.micro
3/3 checks passec
us-east-1a
ec2-100-54
i-05c0ef38b31c5928e (Terraform-Cricbuzz-Server)
Details
Status and alarms
Monitoring
Security
Networking
Storage
Tags
Instance summary Info
Instance ID
i-05c0ef38b31c5928e
IPv6 address
–
Hostname type
Public IPv4 address
100.54.210.134 | open address
Instance state
Running
Private IP DNS name (IPv4 only)
Private IPv4 addresses
172.31.10.100
Public DNS
ec2-100-54-210-134.compute-1.amazonaws.com | open address
Answer private resource DNS name
CloudShell
Agent Toolkit for AWS
Feedback
Console Mobile App
© 2026, Amazon Web Services, Inc. or its affiliates.
Privacy
Terms
Cookie preferences

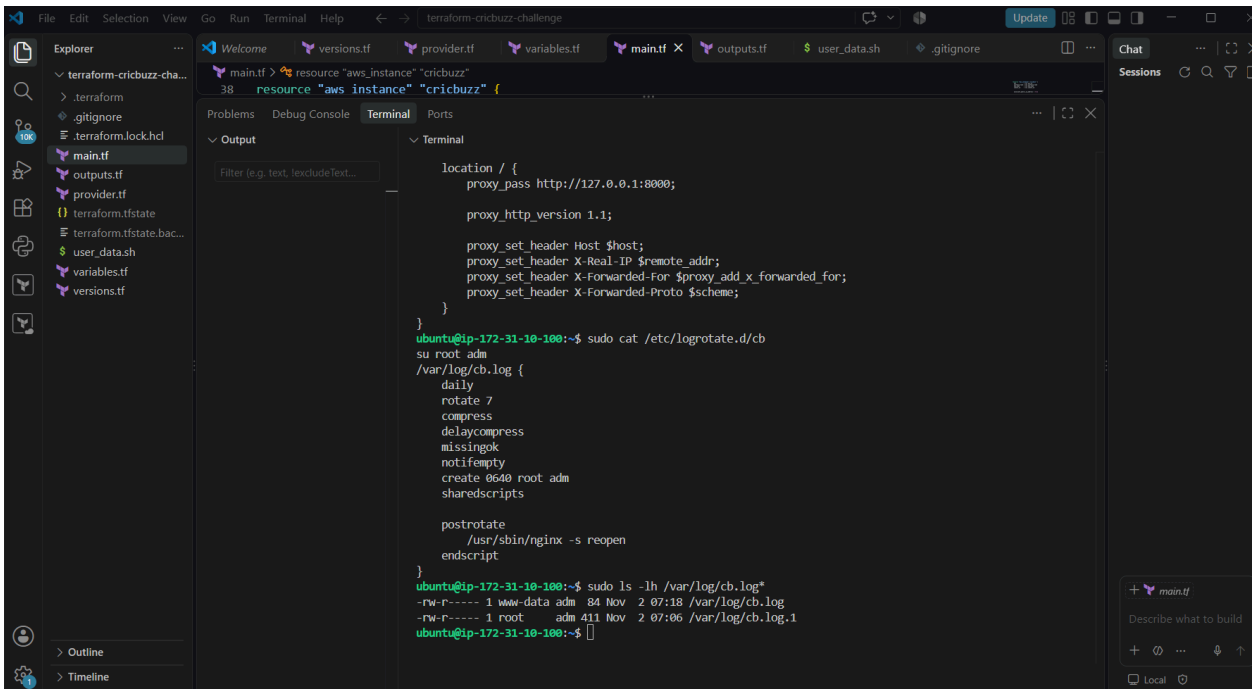

Not secure
100.54.210.134:8080/?utm_source=chatgpt.com
Apps
YouTube
Google
12 Beginner Python...
Learn Python

```
Hello, World!
```


not running postrotate script, since no logs were rotated
switching euid from 0 to 0 and egid from 4 to 0 (pid 3254)
ubuntu@ip-172-31-10-100:~$ sudo logrotate -vf /etc/logrotate.d/cb
reading config file /etc/logrotate.d/cb
Creating stub state file: /var/lib/logrotate/status
acquired lock on state file /var/lib/logrotate/statusReading state from file: /var/lib/logrotate/status
Allocating hash table for state file, size 64 entries
Handling 1 logs
rotating pattern: /var/log/cb.log forced from command line (7 rotations)
empty log files are not rotated, old logs are removed
switching euid from 0 to 0 and egid from 0 to 4 (pid 3257)
considering log /var/log/cb.log
Creating new state
Now: 2026-11-02 07:17
Last rotated at 2026-11-02 07:00
log needs rotating
rotating log /var/log/cb.log, log->rotateCount is 7
dateext suffix '-20261102'
glob pattern '-[0-9][0-9][0-9][0-9][0-9][0-9][0-9][0-9]'
previous log /var/log/cb.log.1 does not exist
renaming /var/log/cb.log.7.gz to /var/log/cb.log.8.gz (rotatecount 7, logstart 1, i 7),
old log /var/log/cb.log.7.gz does not exist
renaming /var/log/cb.log.6.gz to /var/log/cb.log.7.gz (rotatecount 7, logstart 1, i 6),
old log /var/log/cb.log.6.gz does not exist
renaming /var/log/cb.log.5.gz to /var/log/cb.log.6.gz (rotatecount 7, logstart 1, i 5),
old log /var/log/cb.log.5.gz does not exist
renaming /var/log/cb.log.4.gz to /var/log/cb.log.5.gz (rotatecount 7, logstart 1, i 4),
old log /var/log/cb.log.4.gz does not exist
renaming /var/log/cb.log.3.gz to /var/log/cb.log.4.gz (rotatecount 7, logstart 1, i 3),
old log /var/log/cb.log.3.gz does not exist
renaming /var/log/cb.log.2.gz to /var/log/cb.log.3.gz (rotatecount 7, logstart 1, i 2),


switching euid from 0 to 0 and egid from 4 to 0 (pid 3257)
ubuntu@ip-172-31-10-100:~$ sudo ls -lh /var/log/cb.log*
-rw-r----- 1 www-data adm 0 Nov 2 07:17 /var/log/cb.log
-rw-r----- 1 root adm 411 Nov 2 07:06 /var/log/cb.log.1
ubuntu@ip-172-31-10-100:~$ curl http://127.0.0.1:8080/
Hello, World!ubuntu@ip-172-sudo tail -n 5 /var/log/cb.log
127.0.0.1 - - [02/Nov/2026:07:18:32 +0000] "GET / HTTP/1.1" 200 13 "-" "curl/8.5.0"
ubuntu@ip-172-31-10-100:~$ sudo logrotate -d /etc/logrotate.d/cb
warning: logrotate in debug mode does nothing except printing debug messages! Consider using verbose mode
(-v) instead if this is not what you want.
reading config file /etc/logrotate.d/cb
Reading state from file: /var/lib/logrotate/status
Allocating hash table for state file, size 64 entries
Creating new state
Handling 1 logs
rotating pattern: /var/log/cb.log after 1 days (7 rotations)
empty log files are not rotated, old logs are removed
switching euid from 0 to 0 and egid from 0 to 4 (pid 3275)
considering log /var/log/cb.log
Now: 2026-11-02 07:18
Last rotated at 2026-11-02 07:17
log does not need rotating (log has already been rotated)
not running postrotate script, since no logs were rotated
switching euid from 0 to 0 and egid from 4 to 0 (pid 3275)
ubuntu@ip-172-31-10-100:~$ sudo cat /opt/cricbuzz/docker-compose.yml
services:
cricbuzz:
image: ghcr.io/infrastructure-as-code/hello-world:latest
container_name: cricbuzz-hello-world
restart: unless-stopped


File Edit Selection View Go Run Terminal Help
terraform-cricbuzz-challenge
Update
Explorer
terra...
.terraform
.gitignore
.terraform.lock.hcl
main.tf
outputs.tf
provider.tf
terraform.tfstate
terraform.tfstate.bac...
user_data.sh
variables.tf
versions.tf
Welcome
versions.tf
provider.tf
variables.tf
main.tf
outputs.tf
user_data.sh
.gitignore
main.tf > resource "aws_instance" "cricbuzz"
38 resource "aws_instance" "cricbuzz" {
Problems
Debug Console
Terminal
Ports
Output
Filter (e.g. text, !excludeText...
Terminal
switching euid from 0 to 0 and egid from 4 to 0 (pid 3275)
ubuntu@ip-172-31-10-100:~$ sudo cat /opt/cricbuzz/docker-compose.yml
services:
cricbuzz:
image: ghcr.io/infrastructure-as-code/hello-world:latest
container_name: cricbuzz-hello-world
restart: unless-stopped
ports:
- "127.0.0.1:8000:8080"
ubuntu@ip-172-31-10-100:~$ sudo cat /etc/nginx/sites-available/cricbuzz
server {
listen 8080;
server_name _;
access_log /var/log/cb.log;
error_log /var/log/cb-error.log;
location / {
proxy_pass http://127.0.0.1:8000;
proxy_http_version 1.1;
proxy_set_header Host $host;
proxy_set_header X-Real-IP $remote_addr;
proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
proxy_set_header X-Forwarded-Proto $scheme;
}
}
ubuntu@ip-172-31-10-100:~$ sudo cat /etc/logrotate.d/cb
su root adm
/var/log/cb.log {
daily
rotate 7
Outline
Timeline
Chat
Sessions
main.tf
Describe what to build
Local


File Edit Selection View Go Run Terminal Help
terraform-cricbuzz-challenge
Update
Explorer
terraform-cricbuzz-cha...
.terraform
.gitignore
.terraform.lock.hcl
main.tf
outputs.tf
provider.tf
terraform.tfstate
terraform.tfstate.bac...
user_data.sh
variables.tf
versions.tf
Welcome
versions.tf
provider.tf
variables.tf
main.tf
outputs.tf
user_data.sh
.gitignore
main.tf > resource "aws_instance" "cricbuzz"
38 resource "aws_instance" "cricbuzz" {
Problems
Debug Console
Terminal
Ports
Output
Filter (e.g. text, !excludeText...
Terminal
location / {
proxy_pass http://127.0.0.1:8000;
proxy_http_version 1.1;
proxy_set_header Host $host;
proxy_set_header X-Real-IP $remote_addr;
proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
proxy_set_header X-Forwarded-Proto $scheme;
}
}
ubuntu@ip-172-31-10-100:~$ sudo cat /etc/logrotate.d/cb
su root adm
/var/log/cb.log {
daily
rotate 7
compress
delaycompress
missingok
notifempty
create 0640 root adm
sharedscripts
postrotate
/usr/sbin/nginx -s reopen
endscript
}
ubuntu@ip-172-31-10-100:~$ sudo ls -lh /var/log/cb.log*
-rw-r----- 1 www-data adm 84 Nov 2 07:18 /var/log/cb.log
-rw-r----- 1 root adm 411 Nov 2 07:06 /var/log/cb.log.1
ubuntu@ip-172-31-10-100:~$
Outline
Timeline
Chat
Sessions
main.tf
Describe what to build
Local